

Microservices Communications - The Basics

Communication Basics and Differences between Monoliths

Il confronto di partenza

	Monolite	Microservizi
Dove vivono i componenti	Stesso processo applicativo	Processi indipendenti , potenzialmente su server diversi, container diversi, persino località geografiche diverse
Come comunicano	Chiamate a metodo in-process	Protocolli di comunicazione inter-servizio sulla rete
Vantaggi	Veloce, semplice, nessun overhead di rete	Indipendenza, deployment separato
Sfide	Accoppiamento stretto	Latenza, guasti di rete, serializzazione, sicurezza, discovery

"Non è più possibile effettuare chiamate dirette a metodi in-process. Il servizio Basket non può semplicemente istanziare l'Order repository dal servizio Ordering e chiamarne il metodo."

*"Questo passaggio da semplici chiamate di metodo in memoria a comunicazioni inter-servizio basate sulla rete è **un cambiamento di paradigma fondamentale** e definisce le caratteristiche dei microservizi."*

I protocolli principali

Protocollo	Tipo	Uso tipico
HTTP (con API RESTful)	Sincrono	Comunicazione generale request-response
gRPC	Sincrono	RPC ad alte prestazioni
AMQP	Asincrono	Messaggistica via message broker

Perché i servizi devono comunicare

*"I nostri microservizi **non operano in completo isolamento dal punto di vista del business**. Per soddisfare la richiesta di un utente o un processo di business completo, spesso devono collaborare."*

Esempio del checkout:

1. Il client chiama **Basket** per finalizzare il carrello
2. Basket potrebbe **verificare prezzi correnti e livelli di stock** comunicando con **Catalog**
3. Infine comunica con **Ordering** per creare l'ordine

★ Le 5 nuove considerazioni introdotte dalla rete

1. Latenza di rete

*"Ogni chiamata tra servizi comporta ora **un round-trip di rete**, che richiede tempo. Molte chiamate per una singola operazione possono sommarsi e incidere significativamente sul tempo di risposta complessivo per l'utente."*

2. Guasti di rete

*"Le reti non sono perfette. Le connessioni possono cadere. I servizi possono essere temporaneamente non disponibili. **I servizi devono essere progettati per gestire questi errori transitori.**"*

→ risolto nella **Lezione 15** (retry, circuit breaker).

3. Serializzazione / deserializzazione

*"Quando il servizio A chiama il servizio B, gli oggetti complessi devono essere **convertiti in un formato adatto alla trasmissione** (come JSON) e poi **riconvertiti** dal servizio ricevente. **Questo aggiunge overhead di elaborazione.**"*

Questo diventerà il problema che motiva gRPC.

4. Sicurezza

*"Quando i dati viaggiano su una rete — pubblica o anche aziendale — **devono essere protetti**. **HTTPS** per cifrare il traffico è essenziale. Anche **autenticazione e autorizzazione tra servizi** diventano importanti."*

5. Service Discovery ★

*"In un ambiente dinamico dove i microservizi possono **scalare o ridursi**, o le loro istanze possono **spostarsi**, **come fa un servizio a trovare l'indirizzo di rete corrente di un altro servizio che deve chiamare?** Come trova IP e porta?"*

Definizione: Service Discovery — il meccanismo che permette a un servizio di risolvere un **nome logico** (es. catalog) nell'**indirizzo di rete concreto** di un'istanza attiva, senza hardcodare IP e porte. Nel corso è gestito da .NET Aspire in locale; in produzione lo fanno Kubernetes DNS, Consul, Eureka, ecc.

Communication Types: Synchronous or Asynchronous

Comunicazione sincrona (Request-Driven)

Definizione: il servizio A invia una richiesta al servizio B e **attende** — il thread che effettua la richiesta **viene bloccato** — finché B non elabora la richiesta e invia la risposta.

*"Il servizio A resta in attesa e blocca le proprie operazioni **fino a quando riceve la risposta, o fino al timeout se B non risponde.**"*

Protocolli: HTTP/HTTPS (base di REST e GraphQL), **gRPC**.

L'analogia:

"Si può pensare alla comunicazione sincrona come a **una telefonata**: componi il numero, fai la tua domanda, e poi **aspetti in linea** che l'altra persona risponda prima di poter proseguire."

Comunicazione asincrona (Event-Driven)

Definizione: il client invia un messaggio o un evento e **non attende una risposta immediata**. Invia il messaggio ed è **libero di continuare le proprie operazioni**. Il thread **non viene bloccato**.

Tecnologie: message broker come **RabbitMQ o Kafka**.

L'analogia:

"Come **inviare un'email o una lettera**: invii il messaggio e prosegui la tua giornata. **Il destinatario leggerà e agirà quando la riceverà**. Non c'è attesa bloccante."

★ Il confronto completo

	Request-Response (sincrono)	Event-Driven (asincrono)
Protocolli	HTTP/REST, gRPC, GraphQL	Messaging, AMQP
Meccanica	Chiamate dirette ; il client si aspetta un risultato immediato	Il producer emette un evento, i consumer reagiscono
Flusso	Esplicito — il chiamante dirige	Disaccoppiato — nessuna attesa diretta
Vantaggi	Semplice, diretto, immediato	Alto disaccoppiamento, resilienza, scalabilità
Svantaggi	Accoppiamento più stretto , chiamante bloccato, rischio di fallimenti a cascata	Architettura più complessa, difficile da tracciare e debuggare

Sul "**rischio di fallimenti a cascata**": se Basket chiama Ordering sincronamente e Ordering è lento, Basket resta bloccato. Se molti client chiamano Basket, Basket esaurisce i thread. Il guasto di Ordering **si propaga all'indietro** attraverso la catena. È esattamente ciò che i circuit breaker (Lezione 15) e l'asincrono (Lezione 9) risolvono.

Come scegliere

"**Non esiste una risposta univoca**: dipende sempre dalle interazioni specifiche e dai requisiti non funzionali."

La domanda decisiva:

"**Il servizio o il client chiamante ha bisogno di una risposta immediata per continuare il proprio lavoro?**"

Risposta	Scelta	Esempi
Sì	Sincrono	Query, operazioni dove l'utente è attivamente in attesa (es. mostrare la lista prodotti)
No — l'attività può girare in background	Asincrono	"Inviare una notifica email dopo l'invio di un ordine", "aggiornare un sistema secondario"
<i>"Molti processi di business complessi comportano un mix di entrambi gli stili."</i>		

RESTful API Design for Microservices

Definizione

REST = REpresentational State Transfer. Uno **stile architetturale** (non un protocollo) — un insieme di principi guida per progettare applicazioni in rete.

Origine: introdotto da **Roy Fielding** nella sua tesi di dottorato del **2000**, come alternativa **più semplice e flessibile** a SOAP e WSDL.

Caratteristiche:

- Sfrutta **il protocollo HTTP esistente** e ci lavora sopra
- Più veloce ed efficiente delle alternative, con payload **più semplici e minimali**
- Le applicazioni si scambiano **rappresentazioni di risorse** (es. informazioni su un prodotto o un ordine), tipicamente in **JSON**

I vincoli REST (elencati ma non approfonditi nel corso):

- **Stateless** — *"ogni richiesta dal client al server deve contenere tutte le informazioni necessarie a comprenderla"*
- **Interfaccia uniforme**
- **Cacheable**
- **Modello client-server**
- **Sistema a strati (layered system)**
- **Code on demand** (opzionale)

Il Richardson Maturity Model

Definizione: un modello proposto da **Leonard Richardson** che descrive **quanto un'API è realmente RESTful**, su 4 livelli.

Livello	Cosa fa
0	Un solo URI , tutte le operazioni passano da lì (es. SOAP-style: POST /api con l'azione nel body)
1	URI separati per le singole risorse (/products, /orders)
2	Usa i metodi HTTP per definire le operazioni sulle risorse (GET, POST, PUT, DELETE) ← <i>la maggior parte delle API reali si ferma qui</i>
3	Usa gli hypermedia (HATEOAS) — le risposte contengono i link alle azioni possibili

I metodi HTTP fondamentali

Metodo	Uso	Sicuro?	Idempotente?	Note
GET	Recuperare risorse	✓ Sì — <i>"non deve cambiare lo stato del server"</i>	✓ Sì	Di solito senza request body
POST	Creare una nuova risorsa	✗ No	✗ No	Dati nel body . Usabile anche per azioni che non rientrano negli altri verbi
PUT	Aggiornare completamente una	✗ No	✓ Sì	Si invia l'intera rappresentazione

Metodo	Uso	Sicuro?	Idempotente?	Note
	risorsa esistente (o crearla a un URI noto)			
PATCH	Aggiornare parzialmente	✗ No	Dipende	Si inviano solo le modifiche
DELETE	Rimuovere una risorsa	✗ No	✓ Sì	—

Definizione: idempotenza — *"fare la stessa richiesta più volte ha lo stesso effetto che farla una sola volta"*. La risorsa finisce nello stesso stato.

Perché conta enormemente nei microservizi: se una richiesta va in timeout, **non sai se è arrivata**. Se l'operazione è idempotente, **puoi ritentare in sicurezza**. Se non lo è (POST), rischi di creare due ordini. È lo stesso concetto che tornerà con i consumer idempotenti nella Lezione 9.

★ Le 4 regole di design REST

Regola 1 — Progettare risorse (sostantivi), non azioni (verbi)

*"L'URI deve basarsi sui **sostantivi**, cioè sulle **risorse**, non sui verbi. **L'azione è implicita nel metodo HTTP, non nell'URL.**"*

✗ Sbagliato

POST /createProduct
GET /getAllProducts
POST /deleteProduct/5

✓ Corretto

POST /products
GET /products
DELETE /products/5

"Se progettate nomi di azioni, state sbagliando. Progettate sostantivi."

Regola 2 — Ogni istanza di risorsa ha un identificatore univoco nel suo URI

GET /products/5 → recupera il prodotto con id 5

PUT /products/5 → aggiorna il prodotto 5

DELETE /products/5 → elimina il prodotto 5

Regola 3 — Usare JSON per le rappresentazioni

Sia in risposta (GET) sia in richiesta (POST/PUT/PATCH).

Regola 4 — Eseguire le operazioni con i verbi HTTP

POST /products

Body: { "name": "...", "price": ..., "description": ... }

→ Response: 201 Created

Risorse in relazione — la regola anti-annidamento

Il caso semplice: *"gli ordini di un cliente"*

GET /customers/6/orders ✓

Il caso complesso: *"i prodotti dell'ordine 22 del cliente 6"*

GET /customers/6/orders/22/products ✗ SBAGLIATO

"È sbagliato, perché il design è sbagliato. Questo livello di complessità è difficile da mantenere e non è flessibile se le relazioni tra risorse cambiano in futuro. Dovremmo evitare percorsi troppo complessi e profondamente annidati."

★ La best practice:

Gli URI delle risorse non dovrebbero essere più complessi di **/collection/item/collection**.

La soluzione corretta — due chiamate:

- 1) GET /customers/6/orders → ottieni gli ordini, tra cui l'ordine 22
- 2) GET /orders/22/products → ottieni i prodotti di quell'ordine

*L'obiettivo è ottenere **URI chiari, comprensibili e mantenibili**."*

Nota: tieni a mente questa scelta. È corretta dal punto di vista del design REST, **ma costa due round-trip** — ed è esattamente il seme del problema N+1 che esploderà nella sezione successiva.

La tabella completa di design

Endpoint	POST	GET	PUT	DELETE
/customers	Crea un nuovo cliente	Recupera tutti i clienti	⚠️ aggiorna tutti — evitare	⚠️ elimina tutti — evitare
/customers/1	❌ non applicabile	Recupera il cliente 1	Aggiorna il cliente 1	Elimina il cliente 1
/customers/1/orders	Crea un ordine per il cliente 1	Recupera gli ordini del cliente 1	⚠️ evitare	⚠️ evitare

API per dati relazionali tra microservizi

*"In un'architettura a microservizi, i microservizi **non condividono la codebase né i data store**. Invece, **comunicano tramite API** per le operazioni sui dati."*

Esempio concreto:

1. Il servizio **Basket** ha bisogno di informazioni su un cliente
2. Il servizio **Customer** espone GET /api/customer/{id}
3. Basket invia una **HTTP GET** a quell'URL
4. Customer recupera i dati tramite le classi repository e **restituisce l'entità come oggetto JSON** nella risposta HTTP

*"In questo modo si ottiene **l'isolamento dei servizi**, restituendo un'entità risorsa in JSON come risposta alla richiesta HTTP."*

★ API Versioning

Il problema:

*"Se un'API cambia in un microservizio, **le comunicazioni che dipendono da quell'API possono rompersi** in altri microservizi."*

*"Man mano che i microservizi evolvono, servirà **aggiungere funzionalità, correggere bug o modificare la rappresentazione dei dati**. Questo tipo di modifiche può cambiare il contratto dell'API — la struttura di richieste e risposte. Se modifichi un'API da cui dipendono client o servizi esistenti, puoi romperli."*

L'idea:

*"Consentire ai **consumatori esistenti di continuare a usare una versione più vecchia e stabile**, mentre introduci una nuova versione con **breaking changes** per i nuovi client, o per quelli esistenti che possono migrare al proprio ritmo."*

Le 3 strategie:

Strategia	Come	Esempio
URI versioning	Numero di versione nel percorso — "molto comune ed esplicito"	/api/v1/products, /api/v2/products
Header versioning	Il client richiede la versione con un header HTTP personalizzato	api-version: 2
Query parameter versioning	Versione come parametro di query	/products?version=2

I 3 principi guida:

1. **Supportare più versioni affiancate** per un certo periodo
2. **Rendere le modifiche retrocompatibili** quando possibile, all'interno di una versione
3. **Comunicare chiaramente le modifiche** ai consumatori dell'API

Design: E-Shop Microservices with RESTful API Design

Toolkit aggiornato

Nuovi requisiti non funzionali:

- **API ben definite e discoverable**, per una comunicazione chiara tra client, webapp e microservizi
- **Design API stateless**, aderendo ai principi REST

Nuovi pattern nel toolkit: design di API RESTful basate su HTTP — URI orientati alle risorse, uso semantico dei metodi HTTP, JSON per i dati, **API versioning**.

Il design degli endpoint per microservizio

Customer microservice

GET	/customers	→ tutti i clienti
GET	/customers/{id}	→ un cliente specifico
POST	/customers	→ crea cliente
PUT	/customers/{id}	→ aggiorna cliente
DELETE	/customers/{id}	→ elimina cliente

Catalog (Product) microservice

GET /products → tutti i prodotti
GET /products/{id} → un prodotto specifico
POST /products → crea prodotto
PUT /products/{id} → aggiorna prodotto
DELETE /products/{id} → elimina prodotto

Basket microservice — ha endpoint aggiuntivi:

GET /basket → recupera il carrello
GET /basket/{id} → carrello di un utente specifico
POST /basket/{id}/items → aggiungi un articolo
PUT /basket/{id}/items → aggiorna la quantità di un articolo
DELETE /basket/{id} → elimina il carrello
POST /basket/{id}/checkout ★ → CHECKOUT

Nota l'eccezione: /checkout è un verbo, il che apparentemente viola la Regola 1. È l'eccezione pragmatica riconosciuta anche nella pratica REST: alcune **operazioni di business** (checkout, cancel, approve) **non mappano naturalmente su CRUD di una risorsa**. La lezione lo presenta come scelta consapevole: *"possiamo inviare una POST a questo endpoint per identificare specificamente questo endpoint di checkout"*.

Ordering microservice

GET /orders → tutti gli ordini
GET /orders/{id} → un ordine specifico

Un punto architetturale importante

*"I microservizi possono anche **chiamare le rispettive API** per esigenze specifiche. Sebbene il nostro obiettivo sia **minimizzare le chiamate inter-servizio sincrone dirette ove possibile**, il nostro scopo qui è progettare API RESTful chiare, coerenti e intuitive."*

Code: E-Shop Microservices with RESTful API Design

Cartella 7-eshop-microservices-sync-call.

La modifica nell'AppHost

"Il cambiamento chiave per la comunicazione inter-servizio è che il progetto Basket ora fa esplicitamente riferimento al microservizio Catalog."

```
var catalog = builder.AddProject<Catalog>("catalog").WithReference(catalogDb);
```

```
var basket = builder.AddProject<Basket>("basket")
    .WithReference(cache)
    .WithReference(catalog); // ← NUOVO: Basket → Catalog
```

*"Aggiungendo il riferimento, Aspire può **iniettare la configurazione necessaria** — come l'indirizzo del servizio Catalog — dentro Basket, e **consentirgli di effettuare chiamate HTTP** a Catalog."*

Verificabile nella dashboard: nelle **variabili d'ambiente** del microservizio Basket compare un `services__catalog__` . . . — *"che rappresenta il service discovery e la comunicazione tra Basket e Catalog"*.

Come sono esposti gli endpoint

Ogni microservizio ha una cartella `Endpoints/` che espone le API con le **Minimal API di ASP.NET Core**:

```
// Catalog/Endpoints/ProductEndpoints.cs
app.MapGet("/products",      async (ProductService svc) => await svc.GetProducts());
app.MapGet("/products/{id}", async (int id, ProductService svc) => await
svc.GetProductById(id));
app.MapPost("/products",    ...); // create
app.MapPut("/products/{id}", ...); // update
app.MapDelete("/products/{id}", ...); // delete
app.MapGet("/products/search", ...); // search
```

*È qui che esponiamo gli endpoint specifici del microservizio Catalog. **La struttura di questa cartella è la stessa negli altri microservizi.***

🔗 I file `.http` — testare senza Postman

*"Ho incluso anche i file `.http` — `catalog.http`, `basket.http`, `order.http`. Questi file consentono di **inviare richieste HTTP direttamente ai microservizi da Visual Studio**. Non serve usare Postman o altri strumenti esterni."*

È un dettaglio pratico molto utile: i file `.http` (supportati da Visual Studio, VS Code con REST Client, e Rider) contengono richieste in testo semplice, versionabili con il codice:

Get all products

GET `https://localhost:7001/products`

Create a product

POST `https://localhost:7001/products` `Content-Type: application/json`

```
{ "name": "Torch", "price": 29.99, "description": "..." }
```

★ La chiamata sincrona Basket → Catalog

Questa è la parte più interessante della lezione dal punto di vista del codice.

Dove sta: Basket/Services/BasketService.cs, metodo UpdateBasket.

La logica di business:

*"Prima di aggiornare — cioè aggiungere o rimuovere un articolo dal carrello — dobbiamo chiamare il metodo **GetProductById** del microservizio Catalog. Questo per ottenere le informazioni più recenti sul prodotto: impostare il prezzo e il nome del prodotto se sono cambiati."*

Il codice (semplificato):

```
foreach (var item in basket.Items)    // ← per OGNI articolo del carrello
{
    var product = await catalogApi.GetProductById(item.ProductId); // chiamata HTTP
    item.Price = product.Price;    // sovrascrive il prezzo
    item.ProductName = product.Name;
}
await cache.SetStringAsync(basket.UserName, JsonSerializer.Serialize(basket));
```

"Si tratta di una comunicazione sincrona che blocca la chiamata da Basket a Catalog."

⚠ **Chiarimento importante:** la trascrizione dice a volte "comunicazione asincrona" — ma si riferisce all'async/await di C#, che è un dettaglio di **programmazione** (non bloccare il thread). Dal punto di vista **architetturale** questa è **comunicazione sincrona**: Basket non può proseguire finché Catalog non risponde. Sono due significati diversi della stessa parola.

Perché è progettata così: il prezzo nel carrello **non deve essere fidato**. Il client potrebbe inviare un prezzo manipolato. Chiamare Catalog garantisce che **il prezzo autorevole venga dalla fonte di verità**.

⚠ **Ma nota il problema:** foreach + chiamata HTTP dentro il ciclo = **N chiamate di rete per N articoli**. Con 3 articoli sono 3 round-trip. È **letteralmente un N+1**, ed è la premessa del problema della sezione successiva e di quello su gRPC.

I test eseguiti

Test 1 — CRUD su Catalog (tramite catalog.http):

- GET /products → tutti i prodotti in JSON (id, name, description, price, imageUrl)
- GET /products/1 → un singolo prodotto
- POST /products → creato il prodotto id 10, verificabile in coda alla lista
- PUT /products/10 → aggiornato il nome ("Torch") → risposta **204 No Content** = successo
- DELETE /products/10 → il prodotto sparisce dalla lista

Ogni operazione è verificabile nelle **tracce** Aspire, con la relativa query SQL su PostgreSQL.

Test 2 — La chiamata sincrona (tramite `basket.http`):

1. DELETE `/basket/swn` → si pulisce il carrello
2. GET `/basket/swn` → conferma: nessun carrello
3. POST `/basket` con un JSON contenente 2 articoli

*"Questi prezzi [nel JSON] non sono importanti, perché per ogni `ProductId` invieremo una chiamata sincrona a `Catalog` e **sovrascriveremo questi prezzi**"*

- Il **breakpoint nel `foreach` viene colpito** → si vede la chiamata a `Catalog` per ogni item
 - Nella traccia: POST `/basket` → **2 chiamate** a GET `/products/{id}` su `Catalog` → **2 query SQL** → scrittura in Redis
4. Si aggiunge un **terzo articolo** → questa volta la traccia mostra **3 chiamate** a `Catalog`

"Infine aggiorneremo l'intero JSON nella cache Redis distribuita come informazione del carrello."

Nota perché si riscrive tutto il JSON: **Redis è un key-value store**, non un database relazionale. Il carrello è **un'unica chiave** con dentro l'oggetto serializzato. Non puoi aggiornare un singolo item — riscrivi tutto.

Problem: Multiple Requests (N+1) for Retrieving Relational Data"

Il requisito di business

*"Il team di business chiede una **dashboard cliente completa** nell'applicazione web. Deve mostrare: **il profilo del cliente, gli ordini recenti, e per ciascun ordine la lista dei prodotti con nome e prezzo.**"*

*"È una richiesta **molto ragionevole e comune**. Gli utenti vogliono vedere **informazioni correlate presentate insieme** in modo significativo."*

Come si farebbe con REST — il conteggio

1. GET `/customers/{id}` → profilo cliente [1 chiamata]
2. GET `/customers/{id}/orders` → lista **ordini** (5 ordini) [1 chiamata]
3. per OGNI ordine:
GET `/orders/{orderId}/products` → prodotti dell'ordine [5 chiamate]
4. per OGNI prodotto:
GET `/products/{productId}` → dettagli prodotto [N chiamate]

*"Per visualizzare una semplice pagina, il client potrebbe finire per effettuare **1 chiamata a Customer, 1 chiamata a Ordering, 5 chiamate per la lista prodotti degli***

ordini e 3 chiamate per i prodotti — il che significa 15 chiamate. Potenzialmente servono 7 o 17 richieste HTTP separate."

★ Il problema N+1

Definizione: N+1 query problem — *"Il client esegue una prima query per ottenere la lista degli elementi (es. gli ordini), e poi esegue un'ulteriore query per ciascun elemento per recuperare i dati correlati (es. i prodotti di ogni ordine)."*

Conseguenza: $1 + N$ round-trip di rete → **latenza significativa.**

Gli altri 3 problemi

Over-fetching (recupero eccessivo)

*"Un'API REST come `GET /products/{id}` può restituire **tutti i dettagli del prodotto**, ma il client ha bisogno **solo di nome e prezzo** per una vista di riepilogo. I dati extra vengono recuperati ma **non utilizzati**, con conseguente **spreco di banda e di elaborazione**."*

Under-fetching (recupero insufficiente)

*"Un endpoint potrebbe restituire **troppe poche informazioni**, costringendo il client a **effettuare ulteriori chiamate**. Questo porta di nuovo alla comunicazione **chatty**."*

Complessità lato client

*"Il client diventa responsabile dell'**orchestrazione di queste chiamate multiple**. Questo aggiunge **notevole complessità alla codebase del client**."*

La conclusione

*"Sebbene le API RESTful siano **ottime per le interazioni orientate alle risorse**, possono diventare **inefficienti quando i client devono recuperare un grafo complesso di dati correlati che abbracciano più risorse o servizi**."*

Cosa serve:

1. I client devono poter **richiedere esattamente i dati di cui hanno bisogno**
2. Poter **recuperare dati complessi annidati** che coinvolgono più entità correlate
3. **Ridurre il numero di round-trip** verso il backend

GraphQL: A Query Language for APIs

Definizione

GraphQL è (1) un linguaggio di interrogazione per le API e (2) un runtime lato server per soddisfare quelle query usando i dati esistenti.

Origine: sviluppato internamente da **Facebook** nel **2012**, rilasciato pubblicamente nel **2015**.

L'idea centrale:

"Dare ai client il potere di chiedere esattamente ciò di cui hanno bisogno. I client definiscono la struttura dei dati che richiedono in una query, e il server restituisce una risposta JSON che corrisponde esattamente alla struttura richiesta — né più né meno."

Caratteristica tecnica:

"GraphQL usa HTTP per il trasporto, tipicamente un singolo endpoint POST. Si distingue da REST per il modo in cui i dati vengono richiesti e restituiti."

Nota la differenza strutturale con REST: REST ha molti endpoint, ognuno con una forma di risposta fissa. GraphQL ha un endpoint, con una forma di risposta variabile decisa dal client.

Le 3 caratteristiche chiave

1. "Ask for what you need, get exactly that"

"Il client invia una query specificando i campi esatti che gli interessano per ogni risorsa, e il server risponde solo con quei dati. Questo rende le applicazioni più veloci e stabili, perché controllano il payload, evitando over-fetching e under-fetching."

2. "Get many resources in a single request" ★

"Le query GraphQL possono attraversare le relazioni tra oggetti e recuperare dati da più risorse correlate in un unico round-trip. Questo riduce drasticamente il numero di chiamate API rispetto a REST."

Questa è la risposta diretta al problema N+1.

3. "Evolve APIs without versioning"

"Con REST, aggiungere o rimuovere campi da una risorsa porta spesso a versionare l'API per non rompere i client esistenti. Ma lo schema GraphQL consente di aggiungere nuovi campi e tipi senza impattare le query esistenti. I vecchi campi possono essere deprecati ma restare funzionali, dando ai client il tempo di adattarsi. Questo consente un'unica versione evolutiva dell'API."

Perché funziona: un client GraphQL chiede esplicitamente i campi che vuole. Se aggiungi il campo discount, i client vecchi semplicemente non lo chiedono e non se ne accorgono. In REST, aggiungere un campo alla risposta lo manda a tutti.

I 4 concetti fondamentali

1. Schema ★

"È il fondamento dell'API GraphQL. Scritto in GraphQL Schema Definition Language (SDL), descrive tutti i tipi di dati che i client possono interrogare, i campi disponibili su quei tipi, e le relazioni tra di essi. Agisce come un contratto forte tra client e server."

2. Query

"Le richieste che i client inviano per leggere dati. Una query GraphQL è strutturata in modo da rispecchiare la forma della risposta JSON che il client desidera."

3. Mutation

*"Mentre le query servono a **leggere**, le mutation servono a **scrivere** — creare, aggiornare o cancellare record. Sono strutturate in modo simile alle query, **ma** indicano l'intenzione di modificare i dati."*

4. Resolver ★

*"Funzioni che vengono eseguite **sul server**. Ogni campo dello schema è supportato da **un resolver**. Quando arriva una query, il runtime GraphQL **chiama i resolver appropriati per recuperare i dati effettivi** dai sistemi di backend."*

Il flusso completo:

Client invia query/mutation

- Server GraphQL la valida contro lo SCHEMA
- Esegue la query chiamando i RESOLVER
- I resolver interagiscono con database / servizi
- Restituisce i dati nella struttura richiesta

Esempio: query e risposta

Scenario: il client vuole gli ordini, e per ogni ordine id, totale, e la lista prodotti con nome, prezzo e quantità.

```
query {  
  orders {  
    id  
    totalAmount  
    items {  
      quantity  
      product {  
        name  
        price  
      }  
    }  
  }  
}
```

Risposta:

```
{  
  "data": {
```

```

"orders": [
  { "id": 22, "totalAmount": 150.00,
    "items": [
      { "quantity": 2, "product": { "name": "Torch", "price": 29.99 } }
    ]
  }
]
}

```

"La struttura della risposta corrisponde direttamente alla struttura della query, e il client ottiene esattamente ciò che ha chiesto. Né più né meno. Il tutto in un unico ciclo request-response."

Il confronto finale (esempio blog)

Scenario: ottenere un utente, i suoi post e i suoi follower.

	REST	GraphQL
Chiamate	3 — /users/{id}, /users/{id}/posts, /users/{id}/followers	1
Over-fetching	✓ Sì — gli endpoint restituiscono campi non necessari	✗ No
Round-trip	3	1

Design: Microservices Architecture with GraphQL API Design

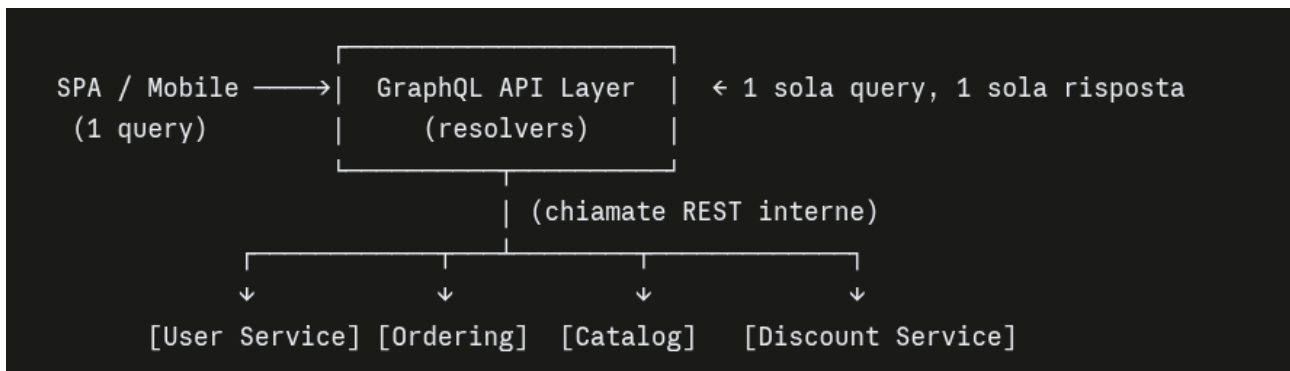
Toolkit aggiornato

Nuovi requisiti non funzionali:

- **Recupero efficiente dei dati** per viste client complesse
- **Risolvere il problema N+1** e **ridurre il comportamento chatty** del client
- **Query guidate dal client** — webapp e mobile possono richiedere **esattamente i dati di cui hanno bisogno per una particolare schermata**
- **Disaccoppiare il client dagli endpoint REST specifici** dei singoli microservizi, soprattutto in aggregazione

Nuovo pattern: design di API GraphQL, *"probabilmente come livello di aggregazione o gateway di aggregazione per le query rivolte ai client"*.

★ **Il design: GraphQL come layer di aggregazione**



Il flusso completo:

1. Il client costruisce **una singola query GraphQL completa**, che descrive **tutte le informazioni annidate** di cui ha bisogno: dettagli cliente → lista ordini → per ogni ordine gli item → per ogni item i dettagli prodotto → per ogni prodotto gli sconti applicabili
2. La query va al **GraphQL API Layer** (un server GraphQL dedicato **o un API Gateway con capacità GraphQL**)
3. **I resolver effettuano le chiamate necessarie ai microservizi downstream:**
 - User Service → dati cliente
 - Ordering Service → ordini del cliente, e poi gli item
 - Catalog Service → dettagli prodotto
 - Discount Service → sconti applicabili
4. **I microservizi downstream continuano a esporre le proprie API REST granulari**, come progettate prima
5. Il layer GraphQL **aggrega** tutte queste informazioni e costruisce **una singola risposta JSON** che corrisponde esattamente alla struttura richiesta
6. **Una sola risposta** torna al client

★ **Il punto chiave da capire: GraphQL non elimina le N chiamate — le sposta.**

Prima il *client* faceva 15 round-trip **attraverso internet**; ora il *GraphQL layer* fa 15 chiamate **dentro il datacenter**, dove la latenza è ordini di grandezza inferiore. **Il client ne fa una sola.**

Scelte tecnologiche

Componente	Opzioni
Client	Libreria GraphQL client per Angular / Vue / React / Blazor
GraphQL layer	Node.js con Apollo Server o Express-GraphQL; ASP.NET con GraphQL.NET (o Hot Chocolate); Java con graphql-java
Microservizi backend	.NET, Java Spring Boot, Python Django... (invariati)
Database	PostgreSQL, Redis, SQL Server... (invariati)

*"Il compito principale di questo layer è **esporre lo schema GraphQL ed eseguire le query chiamando i microservizi di backend.**"*

Problem: Inter-service Communication Makes Heavy Network Traffic

La distinzione fondamentale: API pubbliche vs API interne

Prima del problema, la lezione introduce una distinzione che governerà tutte le scelte successive:

	API pubbliche (Public APIs)	API interne / backend (Internal APIs)
Chi le consuma	Client esterni: webapp, app mobile	Altri microservizi di backend
Esempio	La SPA chiama GET /products	Basket chiama Catalog per i prezzi; Ordering chiama Inventory per conferma stock
Scelta consigliata	REST + JSON su HTTP	"Quando il volume è molto alto o la latenza molto bassa è critica..."
Perché	"JSON è leggibile, ampiamente supportato e facile da usare per vari client"	"La velocità di serializzazione e la dimensione del payload diventano fondamentali"

Lo scenario che rompe REST

*"Immaginate una **flash sale** sul nostro e-shop. **Migliaia di utenti aggiungono contemporaneamente articoli ai carrelli**. Per ogni articolo aggiunto o modificato, Basket effettua una chiamata sincrona a Catalog per il prezzo, e potenzialmente una a Discount per le promozioni."*

*"Se queste chiamate interne usano REST con payload JSON, **l'overhead di serializzazione/deserializzazione JSON e gli header testuali di HTTP/1.1 si sommano in modo significativo, moltiplicati per migliaia di richieste al secondo.**"*

Risultato: **maggiore latenza di rete, maggiore uso di CPU per la serializzazione, tempi di risposta più lenti per l'utente.**

Il problema definito: i 3 limiti di REST/JSON per il traffico interno

1. Dimensione del payload

*"JSON è **basato su testo** e può essere **più prolisso dei formati binari**. Payload più grandi consumano **più banda** e richiedono **più tempo per la trasmissione.**"*

2. Overhead di serializzazione

*"La conversione di oggetti da/verso stringhe JSON può **consumare notevoli risorse di CPU**, soprattutto ad alto throughput."*

3. Limitazioni di HTTP/1.1

*"HTTP/1.1 tradizionale ha limitazioni come **l'head-of-line blocking**, che può impattare le prestazioni **per molte richieste piccole e frequenti**. HTTP/2 migliora le cose."*

Definizione: head-of-line blocking — in HTTP/1.1, su una singola connessione le risposte devono tornare **nell'ordine delle richieste**. Se la prima risposta è lenta, **blocca tutte quelle dietro**, anche se sono già pronte.

Cosa serve

"Il nostro obiettivo è una comunicazione inter-servizio più performante: bassa latenza, alto throughput, payload di dimensioni ridotte, serializzazione veloce e contratti fortemente tipizzati."

gRPC: High Performance Remote Procedure Calls

Definizione

gRPC è un framework open source per chiamate di procedura remota (RPC) ad alte prestazioni, sviluppato inizialmente da Google.

"Progettato specificamente per collegare in modo efficiente i servizi e costruire robusti sistemi distribuiti — un candidato eccellente per la comunicazione microservizio-a-microservizio."

Le 3 caratteristiche tecniche

1. Trasporto: HTTP/2

*"HTTP/2 è una revisione importante di HTTP che offre vantaggi significativi: **framing binario, multiplexing delle richieste su una singola connessione, e compressione degli header.**"*

Il **multiplexing** è ciò che elimina l'head-of-line blocking: più richieste viaggiano in parallelo sulla stessa connessione TCP.

2. Contratti: Protocol Buffers (protobuf)

*"I metodi che un servizio espone e la struttura dei messaggi che scambia sono **definiti usando Protocol Buffers** (file `.proto`)."*

Definizione: Protocol Buffers — *"un meccanismo **neutrale rispetto al linguaggio e alla piattaforma** per la serializzazione di dati strutturati — come XML o JSON, ma **più piccolo, più veloce e più semplice.**"*

3. Generazione automatica del codice

*"Da queste definizioni `.proto`, gli strumenti gRPC possono **generare codice lato client e server** — chiamati **stub e skeleton** — **in molti linguaggi di programmazione diversi.** Questo semplifica lo sviluppo."*

★ Come funziona — i 4 passi

1. Definire il servizio

Si scrive un contratto in un file `.proto`:

```
service DiscountService {  
  rpc GetDiscount (GetDiscountRequest) returns (DiscountModel);  
}
```

```
message GetDiscountRequest {  
    string productName = 1;  
}
```

```
message DiscountModel {  
    int32 id = 1;  
    string productName = 2;  
    int32 amount = 3;  
}
```

2. Generare il codice

*"Si usano gli strumenti di code generation gRPC per creare **stub lato client** e **skeleton lato server**, nel linguaggio scelto: **C#, Java, Python, Go**."*

3. Lato client

*"L'applicazione client interagisce con lo **stub generato**. Quando chiama un metodo su questo stub, è **come se chiamasse un metodo locale**. Ma sotto il cofano, lo stub **serializza i parametri nell'efficiente formato binario protobuf** e invia la richiesta **tramite HTTP/2** al server gRPC."*

4. Lato server

*"Il server gRPC riceve il messaggio binario, **lo deserializza con protobuf** e invoca **l'implementazione del servizio effettiva** — la logica di business che scriviamo. L'implementazione elabora la richiesta e restituisce un messaggio di risposta, **anch'esso serializzato in protobuf** e rispedito via HTTP/2. Lo stub client lo deserializza e lo restituisce all'applicazione."*

*"gRPC **astrae le complessità della comunicazione di rete e della serializzazione** dal punto di vista dello sviluppatore. **È come lavorare con oggetti e metodi locali**, anche se l'esecuzione è distribuita."*

Quando usare gRPC — i 4 casi

Caso	Motivazione
1. Comunicazione interna tra microservizi ★	"Lo scenario principale. Quando i microservizi di backend devono dialogare tra loro con prestazioni elevate, gRPC è spesso la scelta preferita rispetto a REST/JSON "
2. Ambienti poliglotti	"Se avete servizi in linguaggi diversi, il supporto cross-linguaggio via .proto rende la comunicazione perfetta "
3. Ambienti con vincoli di rete	"I messaggi protobuf sono molto più piccoli degli equivalenti JSON → vantaggioso dove la banda è limitata o costosa "
4. Applicazioni real-time / streaming	"Se serve trasmettere grandi quantità di dati o avere comunicazione bidirezionale in tempo reale , le capacità di streaming native di

gRPC sono eccellenti"

★ L'avvertenza importante

*"È importante notare che **gRPC non è destinato a sostituire tutte le API REST/JSON**. Per le API rivolte al pubblico, che devono essere consumate da un'ampia varietà di client esterni — in particolare i browser web — REST e JSON rimangono spesso una scelta molto pratica e accessibile."*

"Ma per il sistema nervoso interno dei microservizi, gRPC offre vantaggi convincenti in termini di prestazioni."

L'applicazione all'e-shop

*"Quando un utente aggiunge un articolo al carrello, Basket deve ottenere **i dettagli più recenti del prodotto da Catalog**, e **le informazioni sugli sconti da Discount**. Attualmente sono chiamate **REST sincrone**. Se queste interazioni diventano **molto frequenti e critiche per le prestazioni**, soprattutto sotto carico elevato — è proprio qui che si possono sostituire le chiamate REST interne con gRPC."*

- **Basket** → client gRPC
- **Catalog e Discount** → espongono **endpoint server gRPC**
- Comunicazione tramite **protobuf su HTTP/2**
- Le API pubbliche esposte da Basket alla webapp restano REST

Design: Microservices Architecture with gRPC API Design

Toolkit aggiornato

Nuovi requisiti non funzionali:

- **Comunicazione interna ad alte prestazioni** — bassa latenza, trasferimento dati efficiente per le comunicazioni critiche
- **Ridurre l'overhead di rete** per il traffico interno

I requisiti precedenti (design REST, deployment indipendente) **restano invariati**.

Nuovo pattern: gRPC API design, *"il nostro nuovo strumento per ottimizzare le comunicazioni interne critiche per le prestazioni"*.

★ L'approccio ibrido — il design finale della lezione



Lato client:

"Il client interagisce con i servizi di backend **principalmente tramite API RESTful su HTTP con payload JSON**. Questo è generalmente **preferito per i client basati su browser**, per facilità d'uso, leggibilità e ampio supporto. **Queste sono le nostre API pubbliche.**"

Lato interno:

"La comunicazione interna avviene **tramite HTTP e gRPC**, soprattutto per **operazioni ad alta frequenza e bassa latenza**. Basket **genera uno stub client per chiamare il metodo RPC** su Discount, che **espone endpoint gRPC definiti con Protocol Buffers**."

Nota importante: un servizio può esporre **entrambi**:

"Oltre alle loro API RESTful pubbliche, i microservizi **possono esporre metodi gRPC**, usando l'approccio di comunicazione ibrido."

★ La best practice riassunta

"Usare API RESTful per la comunicazione client ↔ microservizio (API pubbliche), per la loro ubiquità, facilità di consumo e leggibilità umana. Per la comunicazione interna sensibile alle prestazioni, considerare gRPC per la sua efficienza."

I vantaggi di gRPC internamente

Vantaggio	Dettaglio
Alte prestazioni ★	"Il motore principale. Latenza più bassa e throughput elevato grazie a HTTP/2"
Riduzione della banda	"I payload protobuf sono molto più piccoli delle controparti JSON"
Streaming	"Gestito nativamente ed efficientemente "
Supporto multilinguaggio	"Se scriviamo microservizi in Go o Java per compiti specializzati, gRPC facilita la comunicazione "

★ Gli svantaggi — i 3 trade-off

1. Complessità crescente

"Ora abbiamo **file .proto da definire e gestire**, una **fase di generazione del codice** per gli stub, e **potenzialmente due tipi di endpoint API** — REST per il pubblico e gRPC per l'interno — **da mantenere**."

2. Non leggibile dall'uomo

"Protobuf è un formato binario, quindi non puoi ispezionare facilmente il traffico di rete con strumenti semplici come faresti con JSON. Servono strumenti specializzati per il debug dei messaggi gRPC."

3. Non richiamabile direttamente dal browser

"gRPC non è direttamente richiamabile da un browser web senza un livello intermedio come gRPC-Web, che aggiunge ulteriore complessità. Questo rafforza il motivo per cui REST e JSON sono spesso usati per le API pubbliche."

"Come sempre, ogni scelta progettuale comporta dei trade-off. Usare gRPC internamente ottimizza per prestazioni ed efficienza, al costo di una maggiore complessità di sviluppo."

Problem: Direct Client-to-Service Communication

Il riepilogo

"Finora abbiamo imparato diversi stili di comunicazione sincrona: API RESTful con HTTP e JSON per la comunicazione generale client-servizio, GraphQL per il recupero flessibile dei dati, e gRPC per le prestazioni nella comunicazione interna."

"Ora si tratta di capire come le applicazioni client possano gestire tutti questi protocolli e comunicazioni diverse."

Cosa deve fare il client oggi

Se la webapp chiama **direttamente** più microservizi, deve:

1. **Conoscere gli indirizzi individuali** di molti microservizi diversi
2. **Gestire protocolli e stili di API diversi** — alcune chiamate REST, alcune query GraphQL
3. **Orchestrare chiamate a più servizi** e raccogliere tutti i dati per una singola vista utente
4. **Implementare cross-cutting concern** — autenticazione, logica di retry — **per ciascuna interazione**

"L'applicazione client stessa diventa un complesso punto di fallimento e crea potenziali colli di bottiglia."

Le 4 conseguenze

1. Complessità lato client

"Il codice client deve gestire gli URL di molti servizi, comprendere diversi formati di richiesta/risposta, e spesso aggregare i dati di più chiamate."

2. Interazioni chatty

"Anche se introduciamo GraphQL per l'aggregazione dei dati, il client potrebbe ancora dover effettuare chiamate dirette ad altri servizi REST o gRPC per azioni specifiche, il che comporta più round-trip per una singola operazione utente."

3. Overhead di gestione dei protocolli

*"Se il client deve parlare **REST** con un servizio, **GraphQL** con un altro, e potenzialmente **capire come innescare una chiamata gRPC interna**, deve gestire librerie e logica per tutti questi tipi di protocolli. È molto difficile da gestire in un'unica applicazione client."*

4. ★ **Duplicazione tra client — "un problema enorme"**

*"Immaginiamo che il nostro e-shop abbia **una SPA web, un'app iOS nativa e un'app Android nativa**. Ognuno di questi tre client deve implementare indipendentemente la logica per chiamare e coordinarsi con i nostri microservizi. Sarebbe un'enorme quantità di codice duplicato, uno spreco di sforzi e un incubo di manutenzione."*

*"È un chiaro anti-pattern. È anche una significativa violazione del principio **DRY** (Don't Repeat Yourself)."*

Nota il richiamo esplicito al principio della **Lezione 2** — il corso costruisce continuamente su sé stesso.

5. **Il refactoring del backend rompe i client**

*"Se decidiamo di **dividere un microservizio** o di **cambiarne il protocollo**, ogni **applicazione client che gli parla direttamente** deve essere aggiornata. Questo crea **uno stretto accoppiamento tra i client e la struttura interna del backend**."*

Questo è particolarmente grave con le app mobile: **non puoi forzare gli utenti ad aggiornare**. Una vecchia versione dell'app potrebbe puntare a un servizio che non esiste più.

La soluzione necessaria

*"Questo approccio diretto rende le applicazioni client **fragili, complesse e lente a evolvere**."*

*"Come soluzione, l'applicazione client ha bisogno di **un punto di ingresso al backend più semplice, unificato e gestito**, che **astragga la complessità della struttura di microservizi sottostante**."*

*"Impareremo i pattern di comunicazione dei microservizi, in particolare i **pattern API Gateway e Backend For Frontend (BFF)**."*